```
        }
}
```

5. CẤU TRÚC DỮ LIỆU KIỂU MẢNG (Array)

5.1. Khái niệm về mảng

Mảng là một tập cố định các phần tử cùng có chung một kiểu dữ liệu với các thao tác tạo lập mảng, tìm kiếm, truy cập một phần tử của mảng, lưu trữ mảng. Ngoài giá trị, mỗi phần tử của mảng còn được đặc trưng bởi chỉ số của nó thể hiện thứ tự của phần tử đó trong mảng. Không có các thao tác bổ sung thêm vùng nhớ hoặc loại bỏ vùng nhớ của mảng vì số vùng nhớ cho phần tử trong mảng là cố định.

Một mảng một chiều gồm n phần tử được coi như một vector n thành phần, phần tử thứ i của nó được tương ứng với một chỉ số thứ i - 1 đối với ngôn ngữ lập trình C vì phần tử đầu tiên được bắt đầu từ chỉ số 0. Chúng ta có thể mở rộng khái niệm của mảng một chiều thành khái niệm về mảng nhiều chiều.

Một mảng một chiều gồm n phần tử trong đó mỗi phần tử của nó lại là một mảng một chiều gồm m phần tử được gọi là một mảng hai chiều gồm n x m phần tử.

Tổng quát, một mảng gồm n phần tử mà mỗi phần tử của nó lại là một mảng k - 1 chiều thì nó được gọi là mảng k chiều. Số phần tử của mảng k chiều là tích số giữa số các phần tử của mỗi mảng một chiều.

Khai báo mảmg một chiều được thực hiện theo qui tắc như sau:

```
Tên_kiểu      Tên_biến[Số_phần tử];
```

Ví dụ :

```
int     A[10];  /* khai báo mảng tối đa chứa 10 phần tử nguyên*/
char    str[20]; /* khai báo mảng tối đa chứa 19 kí tự */
float   B[20];  /* khai báo mảng tối đa chứa 20 số thực */
long    int L[20]; /* khai báo mảng tối đa chứa 20 số nguyên dài */
```

b- Cấu trúc lưu trữ của mảng một chiều

Cấu trúc lưu trữ của mảng: Mảng được tổ chức trong bộ nhớ như một vector, mỗi thành phần của vector được tương ứng với một ô nhớ có kích cỡ đúng bằng kích cỡ của kiểu phần tử và được lưu trữ kế tiếp nhau. Nếu chúng ta có khai báo mảng gồm n phần tử thì phần tử đầu tiên là phần tử thứ 0 và phần tử cuối cùng là phần tử thứ n - 1, đồng thời mảng được cấp phát một vùng không gian nhớ liên tục có số byte được tính theo công thức:

Kích_cỡ_mảng = (Số_phần_tử * sizeof (kiểu_phần_tử).

Ví dụ chúng ta có khai báo:

```
int   A[10];
```

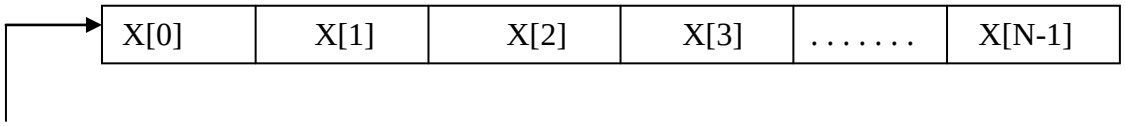
Khi đó kích cỡ tính theo byte của mảng là :

10 *sizeof(int) = 20 byte;

float B[20]; => mảng được cấp phát: 20 * sizeof(float) = 80byte;

Chương trình dịch của ngôn ngữ C luôn qui định tên của mảng đồng thời là địa chỉ phần tử đầu tiên của mảng trong bộ nhớ. Do vậy, nếu ta có một kiểu dữ liệu nào đó là Data_type tên của mảng là X, số phần tử của mảng là 10 thì mảng được tổ chức trong bộ nhớ như sau:

Data_type X[N];



X - là địa chỉ đầu tiên của mảng.
X = &X[0] = (X + 0);
&X[1] = (X + 1);
. . . . . .
&X[i] = (X + i);

Ví dụ: Tìm địa chỉ các phần tử của mảng gồm 10 phần tử nguyên.

```
#include<stdio.h>
#include<conio.h>
void main(void) {
    int     A[10], i ;
/* khai báo mảng gồm 10 biến nguyên */
    printf("\n Địa chỉ đầu của mảng A là : %p", A);
    printf("\n Kích cỡ của mảng   : %5d byte", 10 * sizeof(int));
    for ( i =0 ; i <10; i ++){
        printf("\n Địa chỉ phần tử thứ %5d : %p", i, &A[i]);
    }
    getch();
    }
```

Kết quả thực hiện chương trình:

Địa chỉ đầu của mảng: FFE2

Kích cỡ của mảng: 20

Địa chỉ phần tử thứ 0 = FFE2

Địa chỉ phần tử thứ 1 = FFE4

Địa chỉ phần tử thứ 2 = FFE6

Địa chỉ phần tử thứ 3 = FFE8

PTIT

Địa chỉ phần tử thứ 4 = FFEA

Địa chỉ phần tử thứ 5 = FFEC

Địa chỉ phần tử thứ 6 = FFEE

Địa chỉ phần tử thứ 7 = FFF0

Địa chỉ phần tử thứ 8 = FFF2

Địa chỉ phần tử thứ 9 = FFF4

c- Cấu trúc lưu trữ mảng nhiều chiều

Ngôn ngữ C không hạn chế số chiều của mảng, chế độ cấp phát bộ nhớ cho mảng nhiều chiều được thực hiện theo cơ chế ưu tiên theo hàng.

Khai báo mảng nhiều chiều :

```
Data_type tên_biến[số_chiều_1] [số_chiều_2]. . . [số_chiều_n]
```

Ví dụ:

int A[3][3]; khai báo mảng hai chiều gồm 9 phần tử nguyên được lưu trữ liên tục từ A[0][0] , A[0][1] , A[0][2] , A[1][0] , A[1][1] , A[1][2] , A[2][0] , A[2][1] , A[2][2]

Ví dụ: Kiểm tra cấu trúc lưu trữ của bảng hai chiều trong bộ nhớ.

```
 #include<stdio.h>
#include<conio.h>
void main(void) {
      float    A[3][3] ;
/* khai báo mảng hai chiều gồm 9 phần tử nguyên*/
int i, j;
/* Địa chỉ của các hàng*/
for(i=0; i<3; i++)
      printf("\n Địa chỉ hàng thứ %d là :%p", i, A[i]);
for(i=0; i<3;i++){
      printf("\n");
      for(j=0;j<3;j++)
            printf("%10p",&A[i][j]);
 }
 getch();
}
```

Kết quả thực hiện chương trình:

Địa chỉ hàng thứ 0 = FFD2

Địa chỉ hàng thứ 1 = FFDE

Địa chỉ hàng thứ 2 = FFEA

Địa chỉ phần tử A[0][0]= FFD2

Địa chỉ phần tử A[0][1]= FFD6

Địa chỉ phần tử A[0][2]= FFDA

Địa chỉ phần tử A[1][0]= FFDE

Địa chỉ phần tử A[1][1]= FFE2

Địa chỉ phần tử A[1][2]= FFE6

Địa chỉ phần tử A[2][0]= FFEA

Địa chỉ phần tử A[2][1]= FFEE

Địa chỉ phần tử A[2][2]= FFF2

Ví dụ: Kiểm tra cấu trúc lưu trữ của bảng ba chiều trong bộ nhớ.

```
#include<stdio.h>
#include<conio.h>
void main(void)
{
    float           A[3][4][5] ;
/* khai báo mảng ba chiều */
    int i, j , k;
    for(i=0; i<3;i++){
        for(j=0;j<4;j++){
            printf("\n");
            for( k=0; k <5; k ++)
                printf("%10p",&A[i][j]);
        }
    }
getch();
}
```

Ghi chú: Kết quả thực hiện ví dụ trên có thể cho ra kết quả khác nhau trên các máy tính khác nhau vì việc phân bổ bộ nhớ cho mảng tùy thuộc vào vùng bộ nhớ tự do của mỗi máy.

5.2. Các thao tác đối với mảng

Các thao tác đối với mảng bao gồm: tạo lập mảng, tìm kiếm phần tử của mảng, lưu trữ mảng. Các thao tác này có thể được thực hiện ngay từ khi khai báo mảng. Chúng ta có thể

vừa khai báo mảng vừa khởi đầu cho mảng, song cần chú ý một số kỹ thuật sau khi khởi đầu cho mảng.
Ví dụ:

```
int A[10] = { 5, 7, 2, 1, 9 };
```

Với cách khai báo và khởi đầu như trên, chương trình vẫn phải cấp phát cho mảng A kích cỡ 10 * sizeof(int) = 20 byte bộ nhớ, trong khi đó số byte cần thiết thực sự cho mảng chỉ là 5 * sizeof(int) = 10 byte. Để tránh lãng phí bộ nhớ chúng ta có thể vừa khai báo và khởi đầu dưới dạng sau.

```
int A[] = { 5, 7, 2, 1, 9 };
```

Trong ví dụ này, vùng bộ nhớ cấp phát cho mảng chỉ là số các số nguyên được khởi đầu trong dãy 5 * sizof(int) = 10 byte.

Sau đây là một số ví dụ minh hoạ cho các thao tác xử lý mảng một và nhiều chiều.

Ví dụ: Tạo lập mảng các số thực gồm n phần tử , tìm phần tử lớn nhất và chỉ số của phần tử lớn nhất trong mảng.

```
#include<stdio.h>
#include<conio.h>
#define MAX        100   /*số phần tử tối đa trong mảng*/
void main(void) {
float A[MAX], max; int i, j, n;
/* Khởi tạo mảng số */
printf("\n Nhập số phần tử của mảng n="); scanf("%d", &n);
for(i=0; i<n; i++){
      printf("\n Nhập A[%d] =",i); scanf("%f", &A[i]);
}
max = A[0]; j =0;
for(i=1; i<n; i++){
                if( A[i]>max) {
                        max=A[i]; j = i;
                }
}
printf("\n Chỉ số của phần tử lớn nhất là : %d",j);
printf("\n Giá trị của phần tử lớn nhất là: %6.2f", max);
getch();
}
```

Kết quả thực hiện chương trình:

```
Nhập số phần tử của mảng n=7
Nhap A[0]=1
Nhap A[1]=9
Nhap A[2]=2
Nhap A[3]=8
Nhap A[4]=3
Nhap A[5]=7
Nhap A[6]=4
Chỉ số của phần tử lớn nhất là:      1
Giá trị của phần tử lớn nhất là:     9
```

Ví dụ: Tạo lập ma trận cấp m x n và tìm phần tử lớn nhất, nhỏ nhất của ma trận.

```
#include<stdio.h>
#include<conio.h>
#define     M     20
#define     N     20
void main(void){
float       A[M][N], max, t; int i, j, k, p, m, n;
clrscr();
printf("\n Nhập số hàng của ma trận:"); scanf("%d", &m);
printf("\n Nhập số cột của ma trận:"); scanf("%d", &n);
for(i=0; i<m;i++){
    for(j=0; j<n ; j++){
        printf("\n Nhập A[%d][%d] =", i,j);
        scanf("%f", &t); A[i][j]=t;
    }
}
    max=A[0][0]; k=0; p=0;
    for(i=0; i<m; i++){
        for(j=0;j<n; j++){
            if(A[i][j]>max) {
                max=A[i][j]; k=i ; p =j;
            }
        }
    }
    printf("\n Phần tử có giá trị max là A[%d][%d] = % 6.2f", k,p, max);
    getch();
```

PTIT

```
}
```

Ghi *chú:* C không hỗ trợ khuôn dạng nhập dữ liệu %f cho các mảng nhiều chiều. Do vậy, muốn nhập dữ liệu là số thực cho mảng nhiều chiều chúng ta phải nhập vào biến trung gian sau đó gán giá trị trở lại. Đây không phải là hạn chế của C++ mà hàm scanf() đã được thay thế bởi toán tử "cin". Tuy nhiên, khi sử dụng cin, cout chúng ta phải viết chương trình dưới dạng *.cpp.

5.3. *Mảng và đối của hàm*

Như chúng ta đã biết, khi hàm được truyền theo tham biến thì giá trị của biến có thể bị thay đổi sau mỗi lời gọi hàm. Hàm được gọi là truyền theo tham biến khi chúng ta truyền cho hàm là địa chỉ của biến. Ngôn ngữ C qui định, tên của mảng đồng thời là địa chỉ của mảng trong bộ nhớ, do vậy nếu chúng ta truyền cho hàm là tên của một mảng thì hàm luôn thực hiện theo cơ chế truyền theo tham biến, trường hợp này giống như ta sử dụng từ khoá var trong khai báo biến của hàm trong Pascal. Trong trường hợp muốn truyền theo tham trị với đối số của hàm là một mảng, thì ta phải thực hiện trên một bản sao khác của mảng khi đó các thao tác đối với mảng thực chất đã được thực hiện trên một vùng nhớ khác dành cho bản sao của mảng.

Ví dụ: Tạo lập và sắp xếp dãy các số thực A1, A2, . . . An theo thứ tự tăng dần.

Để giải quyết bài toán, chúng ta thực hiện xây dựng chương trình thành 3 hàm riêng biệt, hàm Init_Array() có nhiệm vụ tạo lập mảng số A[n], hàm Sort_Array() thực hiện việc sắp xếp dãy các số được lưu trữ trong mảng, hàm In_Array() in lại kết quả sau khi mảng đã được sắp xếp.

```
#include<stdio.h>
#include<conio.h>
#define  MAX      100
/* Khai báo nguyên mẫu cho hàm *
void        Init_Array ( float A[], int n);
void        Sort_Array( float A[], int n);
void        In_Array( float A[], int n);
/* Mô tả hàm */
/* Hàm tạo lập mảng số */
void Init_Array( float A[], int n) {
    int      i;
    for( i = 0; i < n; i++ ) {
            printf("\n Nhập A[%d] = ", i);
            scanf("%f", &A[i]);
    }
```

```
    }
    /* Hàm sắp xếp mảng số */
    void Sort_Array( float A[], int n ){
    int i , j ;
    float temp;
    for(i=0; i<n - 1 ; i ++ ) {
                for( j = i + 1; j < n ; j ++ ){
                        if ( A[i] >A[j]) {
                           temp = A[i];  A[i] = A[j]; A[j] = temp;
                        }
                }
    }
  }
    /* Hàm in mảng số */
    void In_Array ( float A[], int n) {
        int  i;
        for(i=0; i<n; i++)
          printf("\n Phần tử A[%d] = %6.2f", i, A[i]);
        getch();
    }
/* Chương trình chính */
void main(void) {
  float   A[MAX]; int  n;
  printf("\n Nhập số phần tử của mảng n = "); scanf("%d", &n);
  Init_Array(A, n);
  Sort_Array(A,n);
  In_Array(A, n);
}
```

Ví dụ: Viết chương trình tính tổng của hai ma trận cùng cấp.

Chương trình được xây dựng thành 3 hàm, hàm Init_Matrix() : Tạo lập ma trận cấp m x n; hàm Sum_Matrix() tính tổng hai ma trận cùng cấp; hàm Print_Matrix() in ma trận kết quả. Tham biến được truyền vào cho hàm là tên ma trận, số hàng, số cột của ma trận.

```
#include      <stdio.h>
#include      <conio.h>
#include      <dos.h>/* khai báo sử dụng hàm delay() trong chương trình*/
#define  M    20      /* Số hàng tối đa của ma trận*/
#define  N    20      /* Số cột tối đa của ma trận */
```

```
/* Khai báo nguyên mẫu cho hàm*/
void  Init_Matrix(float A[M][N], int m, int n, char ten);
void Sum_Matrix(float A[M][N], float B[M][N], float C[M][N], int m, int n);
void Print_Matrix(float A[M][N], int m, int n);
/*Mô tả hàm */
void Init_Matrix(float A[M][N], int m, int n, char ten) {
    int i, j; float temp; clrscr();
     for(i=0; i<m; i++){
          for(j=0; j<n; j++){
                    printf("\n Nhập %c[%d][%d] =", ten, i,j);
                    scanf("%f", &temp); A[i][j]=temp;
          }
     }
   }
  void Sum_Matrix(float A[M][N],float B[M][N], float C[M][N], int m,int n){
          int i, j;
          for(i=0; i<m; i++){
                    for(j=0; j<n; j++){
                              C[i][j]=A[i][j] + B[i][j];
                    }
          }
    }
    void Print_Matrix(float A[M][N], int m, int n) {
   int i, j , ch=179; /* 179 là mã kí tự '|' */
    for(i=0; i<m; i++){
          printf("\n %-3c", ch);
          for(j=0; j<n; j++){
                    printf(" %6.2f", A[i][j];
          }
          printf("%3c", ch);
     }
     getch();
   }
  /* Chương trình chính */
  void main(void) {
    float A[M][N], B[M][N], C[M][N];
          int n, m; clrscr();
          printf("\n Nhập số hàng m ="); scanf("%d", &m);
          printf("\n Nhập số cột n ="); scanf("%d", &n);
          Init_Matrix(A, m, n, 'A');
```

PTIT

```
    Init_Matrix(B, m, n, 'B');
    Sum_Matrix(A, B, C, m, n);
    Print_Matrix(C, m, n);
}
```

5.4. Xâu kí tự (string)

Xâu kí tự là một mảng trong đó mỗi phần tử của nó là một kí tự, kí tự cuối cùng của xâu được dùng làm kí tự kết thúc xâu. Kí tự kết thúc xâu được ngôn ngữ C qui định là kí tự '\0', kí tự này có mã là 0 (NULL) trong bảng mã ASCII. Ví dụ trong khai báo :

```
char str[]='ABCDEF'
```

Khi đó xâu kí tự được tổ chức như sau:

0	1	2	3	4	5	6
A	B	C	D	E	F	'\0'

Khi đó str[0] = 'A'; str[1] = 'B', . ., str[5]='F', str[6]='\0';

Vì kí hiệu kết thúc xâu có mã là 0 nên chúng ta có thể kiểm chứng tổ chức lưu trữ của xâu thông qua đoạn chương trình sau:

Ví dụ:

```
/* In ra từng kí tự trong xâu */
#include    <stdio.h>
#include    <conio.h>
#include    <string.h>
/* sử dụng hàm sử lý xâu kí tự gets() */
void main(void) {
    char str[20]; int i =0; char c;
    printf("\n Nhập xâu kí tự:"); gets(str); /* nhập xâu kí tự từ bàn phím */
    while ( str[i]!='\0'){
        putch(c); i++;
    }
}
```

Ghi chú: Hàm getch() nhận một kí tự từ bàn phím, hàm putch(c) đưa ra màn hình một kí tự c. Hàm sacnf("%s", str) : nhận một xâu kí tự từ bàn phím nhưng không được chứa kí tự trống (space), hàm gets(str) : cho phép nhận từ bàn phím một xâu kí tự kể cả dấu trống.

Ngôn ngữ C không cung cấp các phép toán trên xâu kí tự, mà mọi thao tác trên xâu kí tự đều phải được thực hiện thông qua các lời gọi hàm. Sau đây là một số hàm xử lý xâu kí tự thông dụng được khai báo trong tệp string.h:

puts (string) : Đưa ra màn hình một string.

gets(string) **:** Nhận từ bàn phím một string.
scanf("%s", string): Nhận từ bàn phím một string không kể kí tự trống (space) .
strlen(string): Hàm trả lại một số là độ dài của string.
strcpy(s,p) : Hàm copy xâu p vào xâu s.
strcat(s,p) : Hàm nối xâu p vào sau xâu s.
strcmp(s,p): Hàm trả lại giá trị dương nếu xâu s lớn hơn xâu p, trả lại giá trị âm nếu xâu s nhỏ hơn xâu p, trả lại giá trị 0 nếu xâu s đúng bằng xâu p.

strstr(s,p) : Hàm trả lại vị trí của xâu p trong xâu s, nếu p không có mặt trong s hàm trả lại con trỏ NULL.
strncmp(s,p,n) : Hàm so sánh n kí tự đầu tiên của xâu s và p.
strncpy(s,p,n) : Hàm copy n kí tự đầu tiên từ xâu p vào xâu s.
strrev(str) **:** Hàm đảo xâu s theo thứ tự ngược lại.

Chúng ta có thể sử dụng trực tiếp các hàm xử lý xâu kí tự bằng việc khai báo chỉ thị *#include<string.h>*, tuy nhiên chúng ta có thể viết lại các thao tác đó thông qua ví dụ sau:

Ví dụ: Xây dựng các thao tác sau cho string:

F1- Nhập xâu kí tự từ bàn phím hàm gets(str).
F2- Tìm độ dài xâu kí tự strlen(str).
F3- Tìm vị trí kí tự C đầu tiên xuất hiện trong xâu kí tự.
F4- Đảo xâu kí tự.
F5- Đổi xâu kí tự từ in thường thành in hoa.
F6- Sắp xếp xâu kí tự theo thứ tự tăng dần. . .

```
/* Các thao tác với xâu kí tự */
#include<stdio.h>
#include<conio.h>
#include<dos.h>
#define     F1      59
#define     F2      60
#define     F3      61
#define     F4      62
#define     F5      63
#define     F6      64
#define     F7      65
#define     F10     68
#define     MAX   256
/* khai báo nguyên mẫu cho hàm */
char *gets (char str[]); /* char * được hiểu là một xâu kí tự */
int   strlen(char str[]); /* hàm trả lại độ dài xâu */
int   strcstr(char str[], char c); /* hàm trả lại vị trí kí tự c đầu tiên trong str*/
char *strrev(char str[]);/* hàm đảo xâu str*/
```

PTIT

```
char *upper(char str[]); /* hàm đổi xâu str thành chữ in hoa*/
char *sort_str(char str[]); /* hàm sắp xếp xâu theo thứ tự từ điển*/
void thuc_hien(void);
/* Mô tả hàm */
/* Hàm trả lại một xâu kí tự được nhập từ bàn phím*/
char*gets( char str[] ) {
      int i=0; char c;
      while ( ( c=getch())!='\n') { /* nhập nếu không phải phím enter*/
            str[i] = c; i++;
      }
      str[i]='\0';
      return(str);
}
/* Hàm tính độ dài xâu kí tự: */
int   strlen(char str[]) {
      int i=0;
      while(str[i]) i++;
      return(i);
}
/* Hàm trả lại vị trí đầu tiên kí tự c trong xâu str*/
int   strcstr (char str[] , char c) {
      int i =0;
      while (str[i] && str[i] != c )
            i++;
      if(str[i]=='\0' ) return(-1);
      return(i);
}
/* Hàm đảo xâu kí tự */
char *strrev( char str[]) {
      int i , j , n=strlen(str); char c;
      i = 0; j = n-1;
      while (i < j) {
            c = str[i] ; str[i] = str [j] ; str[j] =c;
      }
      return(str);
}
/* Hàm đổi xâu in thường thành in hoa */
char * upper( char str[] ) {
      int i, n=strlen(str);
      for(i=0;i<n; i++){
            if( str[i]>='a' && str[i]<='z')
                  str[i]=str[i] - 32;
```

PTIT

```
        }
        return(str);
    }
    /* Hàm sắp xếp xâu kí tự */
    char *sort_str( char str[] ) {
        int i, j , n = strlen(str); char temp;
        for (i =0; i<n-1; i++){
                for(j=i+1; j<n; j ++) {
                        if(str[i] >str[j]){
                                temp = str[i]; str[i] = str[j];
                                str[j] = temp;
                        }
                }
        }
    }
    /* Hàm thực hiện chức năng */
    void thuc_hien(void) {
        char c , phim , str[MAX]; int control = 0;
        textmode(0) ;
        do {
                clrscr();
                printf("\n Tập thao tác với string");
                printf("\n F1- Tạo lập string");
                printf("\n F2- Tính độ dài xâu");
                printf("\n F3- Tìm kí tự trong string");
                printf("\n F4- Đảo ngược string");
                printf("\n F5- Đổi thành in hoa");
                printf("\n F6- Sắp xếp string");
                printf("\n F10- Trở về");
                phim = getch();
                switch(phim){
                        case F1: gets(str); control=1; break;
                        case F2:
                                if (control)
                                        printf("\n Độ dài xâu là:%d", strlen(str));
                                break;
                        case F3:
                                if (control){
                                        printf("\n Kí tự cần tìm:");
                                        scanf("%c", &c);
                                        if(strcstr(str, c)>=0)
```

PTIT

```
                                        printf("\n Vị trí:%d",strcstr(str,c));
                        }
                        break;
                case F4:
                        if (control)
                                printf("\n Xâu đảo:%s", strrev(str));
                        break;
                case F5:
                        if (control)
                                printf("\n In hoa:%s", upper(str));
                        break;
                case F6:
                        if (control)
                         printf("\n Xâu được sắp xếp:%s", sort_str(str));
                        break;
        }
        delay(2000);
    } while(phim!=F10);
}
 /* chương trình chính */
void main(void) {
    thuc_hien();
}
```

Mảng các xâu: mảng các xâu là một mảng mà mỗi phần tử của nó là một xâu.

Ví dụ char buffer[25][80] sẽ khai báo mảng các xâu gồm 25 hàng trong đó mỗi hàng gồm 80 kí tự. Ví dụ sau đây sẽ minh hoạ cho các thao tác trên mảng các string.

Ví dụ: Hãy tạo lập mảng các xâu trong đó mỗi xâu là một từ khoá của ngôn ngữ lập trình C. Sắp xếp mảng các từ khoá theo thứ tự từ điển.

Chương trình sau sẽ được thiết kế thành 3 hàm chính: Hàm Init_KeyWord() thiết lập bảng từ khoá, hàm Sort_KeyWord() sắp xếp mảng từ khoá, hàm In_KeyWord() in mảng các từ khoá.

Chương trình được thực hiện như sau:

```
/* Thao tác với mảng các string */
#include<stdio.h>
#include<conio.h>
#include<dos.h>
/* Khai báo nguyên mẫu cho hàm*/
void Init_KeyWord( char key_word[][20], int n);
void Sort_KeyWord(char key_word[][20], int n);
void In_KeyWord(char key_word[][20], int n);
```

PTIT

```
/* Mô tả hàm */
void Init_KeyWord( char key_word[][20], int n) {
    int i;
    for( i = 0; i< n; i++){
        printf("\n Nhập từ khoá %d :",i);
        scanf("%s", key_word[i]);
    }
}
void Sort_KeyWord(char key_word[][20], int n) {
    int i, j; char temp[20];
    for( i = 0; i <n - 1; i++){
        for( j = i +1; j < n; j++){
            if ( strcmp(key_word[i], key_word[j] ) > 0 ){
                strcpy(temp, key_word[i] );
                strcpy(key_word[i], key_word[j] );
                strcpy(key_word[j], temp );
            }
        }
    }
}
void In_KeyWord(char key_word[][20], int n) {
    int i;
    for ( i = 0; i < n; i++){
        printf("\n Key_Word[%d] = %s", i, key_word[i]);
    }
    getch();
}
void main(void) {
    char key_word[100][20]; int n;
    printf("\n Nhập số từ khoá n = "); scanf("%d", &n);
    Init_KeyWord(key_word, n);
    Sort_KeyWord(key_word, n);
    In_KeyWord(key_word, n); }
```

5.5. Kiểu dữ liệu Con trỏ

Con trỏ là biến chứa địa chỉ của một biến khác. Con trỏ được sử dụng rất nhiều trong C, sự mềm dẻo của con trỏ trở thành một thế mạnh để biểu diễn tính toán và truy nhập gián tiếp các đối tượng. Con trỏ đã từng bị coi có hại chẳng kém gì câu lệnh goto vì khi viết chương trình có sử dụng con trỏ sẽ tạo nên các chương trình tương đối khó hiểu. Tuy nhiên, nếu chúng ta có biện pháp quản lý tốt thì dùng con

PTIT

trỏ sẽ làm cho chương trình trở nên đơn giản và góp phần cải thiện đáng kể tốc độ chương trình.

5.5.1 Con trỏ và địa chỉ

Khai báo con trỏ :

Kiểu_dữ_liệu * Biến_con_trỏ;

Vì con trỏ chứa địa chỉ của đối tượng nên có thể thâm nhập vào đối tượng "gián tiếp" qua con trỏ. Giả sử x là một biến, chẳng hạn là biến kiểu int, giả sử px là con trỏ, được tạo ra theo một cách nào đó. Phép toán một ngôi & cho địa chỉ của đối tượng cho nên câu lệnh

px = &x;

sẽ gán địa chỉ của x cho biến px; px bây giờ được gọi là "trỏ tới" x. Phép toán & chỉ áp dụng được cho các biến và phần tử mảng; kết cấu kiểu &(x + 1) và &3 là không hợp lệ.

Phép toán một ngôi * coi toán hạng của nó là địa chỉ cần xét và thâm nhập tới địa chỉ đó để lấy ra nội dung của biến. Ví dụ, nếu y là int thì

y = *px;

sẽ gán cho y nội dung của biến mà px trỏ tới. Vậy dãy

px= &x;

y = *px;

sẽ gán giá trị của x cho y như trong lệnh

y = x;

Cũng cần phải khai báo cho các biến tham dự vào việc này:

int x, y;

int *px;

Khai báo của x và y là điều ta đã biết. Khai báo của con trỏ px có điểm mới

int *px;

có ngụ ý rằng đó là một cách tượng trưng; nó nói lên rằng tổ hợp *px có kiểu int, tức là nếu px xuất hiện trong ngữ cảnh *px thì nó cũng tương đương với biến có kiểu int.

Con trỏ có thể xuất hiện trong các biểu thức. Chẳng hạn, nếu px trỏ tới số nguyên x thì *px có thể xuất hiện trong bất kì ngữ cảnh nào mà x có thể xuất hiện

y = *px + 1; sẽ đặt y lớn hơn x 1 đơn vị;

printf("%d \ n",*px); in ra giá trị hiện tại của x.

phép toán một ngôi * và & có mức ưu tiên cao hơn các phép toán số học, cho nên biểu thức này lấy bất kì giá trị nào mà px trỏ tới, cộng với 1 rồi gán cho y.

Con trỏ cũng có thể xuất hiện bên vế trái của phép gán. Nếu px trỏ tới x thì ***px = 0;** sẽ đặt x thành không và ***px += 1;** sẽ tăng x lên như trong trường hợp **(*px) + +;**

Các dấu ngoặc là cần thiết trong ví dụ cuối; nếu không có chúng thì biểu thức sẽ tăng px thay cho việc tăng ở chỗ nó trỏ tới, bởi vì phép toán một ngôi như * và + + được tính từ phải sang trái.

Cuối cùng vì con trỏ là biến nên ta có thể thao tác chúng như đối với các biến khác. Nếu py là con trỏ nữa kiểu int, thì:

py = px; sẽ sao nội dung của px vào py, nghĩa là làm cho py trỏ tới nơi mà px trỏ. Ví dụ sau minh hoạ những thao tác truy nhập gián tiếp tới biến thông qua con trỏ.

Ví dụ 3.10: Ví dụ sau sẽ thay đổi nội dung của hai biến a và b thông qua con trỏ.

```
#include      <stdio.h>
#include      <conio.h>
void main(void){
        int a = 5, b = 7; /* giả sử có hai biến nguyên a =5, b = 7*/
        int *px, *py;          /* khai báo hai con trỏ kiểu int */
        px = &a;      /* px trỏ tới x */
        printf("\n Nội dung con trỏ px =%d", *px);
        *px  = *px + 10; /* Nội dung của *px là 15*/
        /* con trỏ px đã thay đổi nội dung của a */
        printf("\n Giá trị của a = %d", a);
        px =  &b; /* px trỏ tới b */
        py = px;
        /* con trỏ py thay đổi giá trị của b thông qua con trỏ px*/
        *py  = *py + 10;
        printf("\n Giá trị của b=%d", b);
}
```

Kết quả thực hiện chương trình:

```
        Nội dung con trỏ px : 5
        Giá trị của a : 15
        Giá trị của b : 17
```

PTIT

5.5.2Con trỏ và đối của hàm

Ta sẽ phải làm thế nào nếu phải thay đổi nội dung của đối; Chẳng hạn, chương trình sắp xếp rất có thể dẫn tới việc đổi chỗ hai phần tử chưa theo thứ tự thông qua một hàm là swap. Viết như sau thì chưa đủ

```
swap(a, b);
```

với hàm swap được định nghĩa như sau: swap(x, y) /*sai*/

```
void swap(int x, int y) {
    int temp; temp = x; x = y; y = temp;
}
```

Do việc hàm gọi theo giá trị nên swap không làm ảnh hưởng tới các đối a và b trong hàm gọi tới nó, nghĩa là không thực hiện được việc đổi chỗ. Để thu được kết quả mong muốn, chương trình gọi sẽ truyền con trỏ tới giá trị cần phải thay đổi

```
swap(&a, &b);
```

Vì phép toán & cho địa chỉ của biến, nên &a là con trỏ tới a. Trong bản thân swap, các đối được khai báo là con trỏ, và toán hạng thực tại sẽ thâm nhập qua chúng.

```
void swap(int *px, int *py) {
    int temp;
    temp = *px; *px = *py; *py = temp;
}
```

Con trỏ thường được sử dụng trong các hàm phải cho nhiều hơn một giá trị (có thể nói rằng swap cho hai giá trị, các giá trị mới thông qua đối của nó).

5.5.3 Con trỏ và mảng

Mảng trong C thực chất là một hằng con trỏ, do vậy, mọi thao tác đối với mảng đều có thể được thực hiện thông qua con trỏ.

Khai báo

```
int a[10];
```

xác định mảng có kích thước 10 phần tử int, tức là một khối 10 đối tượng liên tiếp có tên a[0], a[1],...a[9]. Cú pháp a[i] có nghĩa là phần tử của mảng ở vị trí thứ i kể từ vị trí đầu. Nếu pa là con trỏ tới một số nguyên, được khai báo là

```
int *pa;
```

thì phép gán

```
pa = &a[0];
```

sẽ đặt pa để trỏ tới phần tử đầu của a; tức là pa chứa địa chỉ của a[0]. Bây giờ phép gán

```
x = *pa;
```

sẽ sao nội dung của a[0] vào x.

Nếu pa trỏ tới một phần tử của bảng a thì theo định nghĩa, pa + 1 sẽ trỏ tới phần tử tiếp theo và pa -i sẽ trỏ tới phần tử thứ i trước pa, pa + i sẽ trỏ tới phần tử thứ i sau pa. Vậy nếu pa trỏ tới a[0] thì

***(pa + 1)**

sẽ cho nội dung của a[1], pa+i là địa chỉ của a[i] còn *(pa + i) là nội dung của a[i].

Các lưu ý này là đúng bất kể đến kiểu của biến trong mảng a. Định nghĩa "cộng 1 vào con trỏ" sẽ được lấy theo tỉ lệ kích thước lưu trữ của đối tượng mà pa trỏ tới. Vậy trong pa + i, i sẽ được nhân với kích thước của đối tượng mà pa trỏ tới trước khi được cộng vào pa.

Sự tương ứng giữa việc định chỉ số và phép toán số học trên con trỏ là rất chặt chẽ. Trong thực tế, việc tham trỏ tới mảng được trình biên dịch chuyển thành con trỏ tới điểm đầu của mảng. Kết quả là tên mảng chính là một biểu thức con trỏ. Vì tên mảng là đồng nghĩa với việc định vị phần tử thứ 0 của mảng a nên phép gán

pa = &a[0];

cũng có thể viết là

pa = a;

Điều cần chú ý là để tham trỏ tới a[i] có thể được viết dưới dạng *(a + i). Trong khi tính a[i], C chuyển tức khắc nó thành *(a + i); hai dạng này hoàn toàn là tương đương nhau. Áp dụng phép toán & cho cả hai dạng này ta có &a[i] và (a + i) là địa chỉ của phần tử thứ i trong mảng a. Mặt khác nếu pa là con trỏ thì các biểu thức có thể sử dụng nó kèm thêm chỉ số: pa[i] đồng nhất với *(pa + i). Tóm lại, bất kì một mảng và biểu thức chỉ số nào cũng đều được viết như một con trỏ.

Có một sự khác biệt giữa tên mảng và con trỏ cần phải nhớ. Con trỏ là một biến, nên pa = a và pa ++ đều là các phép toán đúng, còn tên mảng là một hằng chứ không là biến: kết cấu kiểu a = pa hoặc a + + hoặc p = &a là không hợp lệ.

Khi truyền một tên mảng cho hàm thì tên của mảng là địa chỉ đầu của mảng, do vậy, tên mảng thực sự là con trỏ. Ta có thể dùng sự kiện này để viết ra bản mới của strlen, tính chiều dài của xâu kí tự.

```
int strlen( char * s) /*cho chiều dài của xâu s*/
{
    int n;
    for (n = 0; *s ! = '\ 0'; s + +)
        n + +;
    return(n);
 }
```

Việc tăng s là hoàn toàn hợp pháp vì nó là biến con trỏ; s + + không ảnh hưởng tới xâu kí tự trong hàm gọi tới strlen mà chỉ làm tăng bản sao riêng của địa chỉ trong strlen.

Vì các tham biến hình thức trong định nghĩa hàm

char s[ ]; và **char *s; l**à hoàn toàn tương đương. Khi truyền một tên mảng cho hàm, hàm có thể coi rằng nó xử lí hoặc mảng hoặc con trỏ là giống nhau. Một khía cạnh khác, nếu p và q trỏ tới các thành phần của cùng một mảng thì có thể áp dụng được các quan hệ như <, < =, > = v.v... chẳng hạn

p < q

là đúng, nếu p con trỏ tới thành phần đứng trước thành phần mà q trỏ tới trong mảng. Các quan hệ = = và != cũng áp dụng được. Có thể so sánh một con trỏ với giá trị NULL. Nhưng so sánh các con trỏ trỏ tới các mảng khác nhau sẽ không đưa lại kết quả mong muốn.

Tiếp nữa, chúng ta đã quan sát rằng con trỏ và số nguyên có thể cộng hoặc trừ cho nhau. Kết cấu

p + n

nghĩa là đối tượng thứ n sau đối tượng do p trỏ tới. Điều này đúng với bất kể loại đối tượng nào mà p được khai báo sẽ trỏ tới; trình biên dịch sẽ tính tỉ lệ n theo kích thước của các đối tượng do p trỏ tới, điều này được xác định theo khai báo của p. Chẳng hạn trên PC AT, nhân từ tỉ lệ là 1 đối với char, 2 đối với int và short, 4 cho long và float.

Phép trừ con trỏ cũng hợp lệ; nếu p và q trỏ tới các thành phần của cùng một mảng thì p - q là số phần tử giữa p và q. Dùng sự kiện này ta có thể viết ra một bản khác cho strlen:

```
/*cho độ dài xâu s*/
int strlen( char *s) {
      char *p = s;
      while (*p ! = '\ 0')
            p + +;
      return(p-s);
 }
```

Trong khai báo, p được khởi đầu là s, tức là trỏ tới kí tự đầu tiên. Chu trình while, kiểm tra lần lượt từng kí tự xem đã là '\ 0' chưa để xác định cuối xâu. Vì '\ 0' là 0 và vì while chỉ kiểm tra xem biểu thức có là 0 hay không nên có thể bỏ phép thử tường minh, vậy ta có thể viết lại chu trình trên

```
while (*p)
      p + +;
```

Do p trỏ tới các kí tự nên p + + sẽ chuyển p để trỏ sang kí tự tiếp từng lúc, và p - s sẽ cho số các kí tự đã lướt qua, tức là độ dài của xâu. Ví dụ sau minh hoạ rõ ràng về phương pháp sử dụng con trỏ.

Ví dụ 3.11- Viết chương trình sắp xếp dãy các số nguyên theo thứ tự tăng dần bằng con trỏ.

Chương trình được thiết kế thành 3 hàm chính, hàm Init_Array() tạo lập dãy các số thực, hàm Sort_Array() sắp xếp dãy các số thực theo thứ tự tăng dần, hàm In_Array() in kết quả đã được sắp xếp.

```
#include      <stdio.h>
#include      <conio.h>
#define       MAX         100
/* Khai báo nguyên mẫu cho hàm */
void Init_Array( float  *A, int  n);
void Sort_Array( float  *A, int  n);
void In_Array( float  *A, int  n);
/* Mô tả hàm*/
void Init_Array( float  *A, int  n) {
        int i;
        for(i=0; i<n;i++){
                printf("\n Nhập A[%d] = ", i);
                scanf("%f", (A + i) );
        }
}
void Sort_Array( float  *A, int  n) {
        int i, j; float temp;
        for ( i =0; i< n -1; i++){
                for( j = i +1; j <n; j++){
                        if( *(A+i) > *(A + j) ) {
                                temp = *( A +i );
                                *(A + i ) = * (A + j );
                                *(A +j) = temp;
                        }
                }
        }
}
void In_Array( float  *A, int  n) {
        int i;
        for(i=0; i<n;i++){
                printf("\n Phần tử A[%d] = % 6.2f", i, *(A +i) );
```

PTIT

```
        }
}
/* chương trình chính */
void main(void) {
        float   A[MAX]; int n;
        printf("\n Nhập n="); scanf("%d", &n);
        Init_Array(A,n);
        Sort_Array(A,n);
        In_Array(A,n);
}
```

5.5.4Cấp phát bộ nhớ cho con trỏ

Nếu p là con trỏ thì p + + sẽ tăng p để trỏ tới phần tiếp của đối tượng mà p đã trỏ tới , bất kể kiểu đối tượng là gì, còn p + = i sẽ tăng lên trỏ tới phần tử thứ i kể từ sau phần tử p trỏ tới. Đa số các trường hợp chúng ta đang xét thì p được trỏ tới một mảng nào đó đã được khai báo trước, khi đó mặc nhiên vùng bộ nhớ mà con trỏ xử dụng là vùng bộ nhớ đã được cấp phát cho mảng trước đó và ta có thể xử lý như một mảng. Tuy nhiên, chúng ta có thể sử dụng con trỏ linh động hơn bằng các phương pháp cấp phát bộ nhớ động cho con trỏ.

Trong thư viện chuẩn của C cung cấp cho ta một số hàm cấp phát bộ nhớ cho con trỏ

Hàm **malloc(size_t size)** : cấp phát một vùng bộ nhớ có kích cỡ size cho con trỏ. Hàm trả lại con trỏ NULL nếu size = 0;

Hàm ***calloc(n , i)*** : đặt con trỏ vào đầu vùng bộ nhớ gồm n x i bytes tự do. Hàm trả lại con trỏ NULL nếu việc cấp phát không thành công.

Hàm farmalloc(), farcalloc() : đặt con trỏ vào đầu vùng bộ nhớ tự do ngoài vùng heap, trả lại con trỏ NULL nếu việc cấp phát không thành công.

Hàm free (p) : giải phóng vùng bộ nhớ đã cấp phát bởi malloc() hoặc calloc() trước đó.

Hàm farfree() : giải phóng vùng bộ nhớ đã cấp phát bởi farmalloc() hoặc farcalloc() trước đó.

Ví dụ 3.12: Xây dựng các thao tác tạo lập đa thức, tính giá trị đa thức theo lược đồ Hooneir, tính tổng hai đa thức, hiệu hai đa thức.

/* Chương trình xây dựng tập thao tác cho đa thức */

```
#include <stdio.h>
#include <dos.h>
#include <conio.h>
#include <alloc.h>
```

```
#define     F1    59
#define     F2    60
#define     F3    61
#define     F4    62
#define     F5    63
#define     F10   68
/* Nguyên mẫu của hàm */
void Init_Dathuc(float *A, int n, char c);
void In_Dathuc(float *A, int n);
float  Val_Dathuc( float *A, int n);
void Tong_Dathuc(float *C, float *A, float *B, int n, int m);
void Hieu_Dathuc(float *C,float *A, float *B, int n, int m);
void Thuc_Hien(void);
/* Mô tả hàm */
void Init_Dathuc(float *A, int n, char c){
        int i;float t;
        for(i=0;i<n;i++){
                printf("\n %c[%d]=",c,i);
                scanf("%f",&t);
                A[i]=t;
        }
        In_Dathuc(A,n);
}
void In_Dathuc(float *A, int n){
        int i;
        for(i=0;i<n;i++)
                printf("%7.2f",A[i]);
        printf("\n");
        delay(2000);
}
float  Val_Dathuc( float *A, int n){
        float s=A[n-1], k; int i, j;
        printf("\n Nhap k="); scanf("%f",&k);
```

PTIT

```
        for(i=n-2; i>=0; i--)
                s = s*k + A[i];
        printf("\n S=%6.2f", s); delay(2000);
        return(s);
}
void Tong_Dathuc(float *C, float *A, float *B, int n, int m){
        int k,i,j;
        if(n>m){
                k=n;
                for(i=0;i<m;i++)
                        C[i]=A[i]+B[i];
                for(i=m; i<n; i++)
                        C[i]=A[i];
        }
        else {
                k=m;
                for(i=0;i<n;i++)
                        C[i]=A[i]+B[i];
                for(i=n; i<m; i++)
                        C[i]=B[i];
        }
        printf("\n Da thuc tong:");
        In_Dathuc(C,k);
}
void Hieu_Dathuc(float *C,float *A, float *B, int n, int m){
        int k,i,j;
        if(n>m){
                k=n;
                for(i=0;i<m;i++)
                        C[i]=A[i]-B[i];
                for(i=m; i<n; i++)
                        C[i]=A[i];
        }
```

PTIT

```
        else {
                k=m;
                for(i=0;i<n;i++)
                        C[i]=A[i]-B[i];
                for(i=n; i<m; i++)
                        C[i]=B[i];
        }
        printf("\n Da thuc hieu:");
        In_Dathuc(C,k);
}
void Thuc_Hien(void){
        float *A, *B, *C;
        int n, m, control = 0;char phim;
        printf("\n Bac cua da thuc A:");scanf("%d", &n);
        printf("\n Bac cua da thuc B:");scanf("%d", &m);
        n=n+1;m=m+1;
        A = malloc( n * szeof(float));
        B = malloc( m * szeof(float));
        C = malloc( (m + n) * szeof(float));
        do {
                clrscr();
                printf("\n Tập thao tác trên đa thức");
                printf("\n F1- Tạo lập đa thức");
                printf("\n F2- Tính giá trị đa thức");
                printf("\n F3- Tổng hai đa thức");
                printf("\n F4- Hiệu hai đa thức");
                printf("\n F10- Tạo lập đa thức");
                phim = getch();
                switch(phim) {
                        case F1: control = 1;
                                Init_Dathuc(A, n, 'A');
                                Init_Dathuc(B, m, 'B'); break;
                        case F2:
```

PTIT

```
                    if(control) {
                        Val_Dathuc(A,n);
                        Val_Dathuc(B, m);
                    }
                    break;
                case F3:
                    if(control)
                        Tong_Dathuc(C, A, B, n , m);
                    break;
                case F4:
                    if(control)
                        Hieu_Dathuc(C, A, B, n , m);
                    break;
            }
        } while( phim !=F10);
        free(A);    free(B);    free(C);
}
void main(void){
        Thuc_Hien();
}
```

Con trỏ với mảng nhiều chiều

C không hạn chế số chiều của mảng nhiều chiều mặc dầu trong thực tế có khuynh hướng là ít sử dụng chúng hơn là mảng con trỏ. Trong mục này ta sẽ đưa ra mối liên hệ giữa con trỏ và mảng nhiều.

Khi xử lý với các mảng nhiều chiều, thông thường lập trình viên thường đưa ra khai báo floatA[3][3]; hoặc float A[N][N] với N được định nghĩa là cực đại của cấp ma trận vuông mà chúng ta cần giải quyết. Để tạo lập ma trận, chúng ta cũng có thể xây dựng thành hàm riêng :

```
Init_Matrix( float A[N][N], int n);
```

hoặc có thể khởi đầu trực tiếp ngay sau khi định nghĩa:

```
float   A[3][3] = {
                    {1   , 2   , 4},
                    {4   , 8   , 12},
                    {3   , -3  , 0 }
            }
```

```
hoặc    A[][3] = {
                    {1    , 2    , 4},
                    {4    , 8    , 12},
                    {3    , -3   , 0 }
                }
```

Cả hai cách khởi đầu đều bắt buộc phải có chỉ số cột, tuy nhiên để thấy rõ được mối liên hệ giữa mảng nhiều chiều và con trỏ chúng ta phải phân tích kỹ lưỡng cấu trúc lưu trữ của bảng nhiều chiều.

Trong ví dụ 3.2 đã chỉ ra cấu trúc lưu trữ của mảng hai chiều theo chế độ ưu tiên hàng. Tên của mảng là địa chỉ của mảng đó trong bộ nhớ điều đó dẫn tới những kết quả như sau:

Địa chỉ của ma trận A[3][3] là A đồng thời là địa chỉ hàng thứ 0 đồng thời là địa chỉ của phần tử đầu tiên của ma trận:

Địa chỉ đầu của ma trận A là A = A[0] = *(A + 0) = & A[0][0] ;

Địa chỉ hàng thứ nhất: A[1] = *(A + 1) = &A[1][0];

Địa chỉ hàng thứ i : A[i] = *(A+i) = &A[i][0];

Địa chỉ phần tử A[i][j] = (*(A+i)) + j;

Nội dung phần tử A[i][j] = *(*(A+i)) + j);

Ví dụ 3.13: Kiểm chứng lại mối quan hệ giữa mảng nhiều chiều với con trỏ thông qua cấu trúc lưu trữ của nó trong bộ nhớ:

```
#include    <stdio.h>
#include    <conio.h>
void main(void) {
    float   A[3][3] ; /* khai báo mảng hai chiều gồm 9 phần tử nguyên*/
    int i, j;
    /* Địa chỉ của các hàng*/
    for(i=0; i<3; i++)
        printf("\n Địa chỉ hàng thứ %d là :%p", i, A[i]);
    /* Địa chỉ hàng truy nhập thông qua con trỏ*/
    printf("\n Truy nhập bằng con trỏ :");
    for(i=0; i<3; i++)
        printf("\n Địa chỉ hàng thứ %d là :%p", i, *(A+i) );
    /*Địa chỉ các phần tử */
    for(i=0; i<3;i++){
        printf("\n");
```

```
            for(j=0;j<3;j++)
                printf("%10p",&A[i][j]);
        }
        /*Địa chỉ các phần tử truy nhập thông qua con trỏ*/
        printf("\n Truy nhập bằng con trỏ");
        for(i=0; i<3;i++){
            printf("\n");
            for(j=0;j<3;j++)
                printf("%10p", ( *( A+i ) ) + j );
        }

        getch();
}
```

Kết quả thực hiện chương trình:

```
        Địa chỉ hàng thứ  0 =      FFD2
        Địa chỉ hàng thứ  1 =      FFDE
        Địa chỉ hàng thứ  2 =      FFEA

        Truy nhập bằng con trỏ:

        Địa chỉ hàng thứ  0 =      FFD2
        Địa chỉ hàng thứ  1 =      FFDE
        Địa chỉ hàng thứ  2 =      FFEA
        Địa chỉ phần tử A[0][0]=    FFD2
        Địa chỉ phần tử A[0][1]=    FFD6
        Địa chỉ phần tử A[0][2]=    FFDA
        Địa chỉ phần tử A[1][0]=    FFDE
        Địa chỉ phần tử A[1][1]=    FFE2
        Địa chỉ phần tử A[1][2]=    FFE6
        Địa chỉ phần tử A[2][0]=    FFEA
        Địa chỉ phần tử A[2][1]=    FFEE
        Địa chỉ phần tử A[2][2]=    FFF2
        Truy nhập bằng con trỏ:
```

PTIT

Địa chỉ phần tử A[0][0]= FFD2

Địa chỉ phần tử A[0][1]= FFD6

Địa chỉ phần tử A[0][2]= FFDA

Địa chỉ phần tử A[1][0]= FFDE

Địa chỉ phần tử A[1][1]= FFE2

Địa chỉ phần tử A[1][2]= FFE6

Địa chỉ phần tử A[2][0]= FFEA

Địa chỉ phần tử A[2][1]= FFEE

Địa chỉ phần tử A[2][2]= FFF2

Như vậy, truy nhập mảng nhiều chiều thông qua các phép toán của mảng cũng giống như truy nhập của mảng nhiều chiều thông qua con trỏ. C cung cấp cho người sử dụng một khai báo tương đương với mảng nhiều chiều đó là thay vì khai báo float A[3][3] ; chúng ta có thể đưa ra khai báo sau:

float (*A)[3];

khai báo một con trỏ kiểu float có thể trỏ tới bảng gồm 3 phần tử float. Như vậy, những hàm được truyền vào các mảng nhiều chiều như :

Init_Matrix(float A[N][N], int n);

có thể được thay thế bằng:

Init_Matrix(float (*A)[N], int n);

Dấu (*A) là cần thiết vì dấu [] có thứ tự ưu tiên cao hơn dấu * và khi đó chương trình dịch sẽ hiểu float *A[N] thành một mảng gồm N con trỏ.

Ví dụ 3.14: Tìm định thức của ma trận vuông cấp n.

Để tính định thức ma trận vuông cấp n, chúng ta sử dụng phương pháp đưa ma trận vuông về dạng đường chéo, chương trình được thiết kế thành các hàm sau:

Hàm Init_Matrix() : tạo lập ma trận vuông A cấp n.

Hàm Chuyen() : chuyển hàng thứ i nếu như A[i][i]=0 thành hàng thứ j <i có A[j][i]<>0.

Hàm Det_Matrix(): chuyển ma trận A cấp n về dạng đường chéo và tính định thức ma trận.

```
#include    <stdio.h>
#include    <conio.h>
#include    <dos.h>
#define             N    10
/* Khai báo nguyên mẫu cho hàm*/
void   Init_Matrix(float (*A)[N], int n);
void   In_Matrix(float (*A)[N], int n);
```

```
int             Chuyen(float (*A)[N], int n, int i);
float   Det_Matrix(float (*A)[N], int n);
/* Mô tả hàm */
void    Init_Matrix( float (*A)[N], int n) {
        int i, j; float temp;
        for(i=0;i<n;i++){
                for(j=0;j<n; j++){
                        printf("\n Nhập A[%d][%d]=", i,j);
                        scanf("%f",&temp);
                        A[i][j]=temp;
                }
        }
        In_Matrix(A,n);
}

void    In_Matrix( float (*A)[N], int n){
        int i, j;
        for(i=0;i<n;i++){
                printf("\n%-3c", 179);
                for(j=0;j<n; j++){
                        printf("%8.2f", A[i][j]);
                }
                printf("%3c", 179);
        }
        delay(2000);
}
int chuyen(float (*A)[N], int n, int i){
        float j, k, temp;
        for(j=i+1; j<n;j++){
                if(A[j][i]){
                        for(k=0;k<n;k++){
                                temp=A[i][k];A[i][k]=A[j][k];
                                A[j][k]=temp;
```

PTIT

```
                    }
                    return(1);
                }
        }
        return(0);
}
float Det_Matrix(float (*A)[N], int n){
        float D=1, p, x; int i, j, k;
        for(i=0; i<n;i++){
                if(A[i][i]==0){
                        if(chuyen(A,n,i)==0){
                                printf("\n Định thức D= 0");
                                delay(2000);
                                return(0);
                        }
                        else
                                D =-D;
                }
                for(j=i+1;j<n;j++){
                        x = A[j][i]/A[i][i];
                        for(k=0; k<n;k++){
                                p = A[i][k] * x;
                                A[j][k]=A[j][k]-p;
                        }
                        In_Matrix(A,n); clrscr();
                }
        }
        for(i=0;i<n; i++)
                D *=A[i][i];
        printf("\n D=%6.2f", D);
        getch(); return(D);
}
void main(void){
```

PTIT

```
        float (*A)[N]; int n;clrscr();
        printf("\n Nhập cấp n="); scanf("%d", &n);
        Init_Matrix(A,n);
        Det_Matrix(A,n);
}
```

Con trỏ trỏ tới con trỏ

Như đã được chỉ ra trong ví dụ 3.13, 3.14. Một mảng được coi như một hằng con trỏ, trong khai báo :

float A[10]; hoàn toàn có thể được thay thế bằng việc sử dụng con trỏ cùng với phép cấp phát bộ nhớ cho con trỏ:

```
float *A;
A = malloc(10 *sizeof(float));
```

đối với mảng nhiều chiều thì khai báo:

```
float A[3][3];
```

có thể được thay thế bằng con trỏ thông qua khai báo:

```
float (*A)[3];
```

Nhưng chúng ta lại nhận thấy:

A[i][j] = (*(A + i))[j] = *((* (A + i)) + j);

Hay nói cách khác, truy nhập tới phần tử A[i][j] được thông qua một con trỏ trỏ tới một con trỏ. Điều này có nghĩa là ta hoàn toàn có thể thay thế một mảng nhiều chiều bằng một con trỏ trỏ tới con trỏ.

Tóm lại, những khai báo sau cho đối của hàm là tương đương:

float A[3][3]; <=> float (*A)[3]; <=> float **A;

Ví dụ sau sẽ minh hoạ cho việc thay thế mảng nhiều chiều bằng con trỏ trỏ tới con trỏ:

Ví dụ 3.15: Kiểm tra tính tương đương giữa mảng nhiều chiều và con trỏ trỏ tới con trỏ.

```
#include    <stdio.h>
#include    <conio.h>
#include    <alloc.h>
void main(void){
    float   **A; /* khai báo con trỏ trỏ tới con trỏ kiểu float */
    int     i, j, n, m;
    printf("\n Nhập số hàng của ma trận m ="); scanf("%d", &m);
    printf("\n Nhập số cột của ma trận n ="); scanf("%d", &n);
```

```
        for ( i =0; i <m; i++)
                A[i] = malloc( (n-1) * sizeof(float) );
        /* Tìm địa chỉ của từng phần tử */
        for(i=0;i<m; i++){
                printf("\n");
                for(j=0;j<n;j++){
                        printf("%10p", &A[i][j]);
                }
        }
        free(A); getch();
}
```

Kết quả thực hiện chương trình:

```
        Nhập số hàng của ma trận m = 3
        Nhập số cột của ma trận n = 3
          073E    0742    0746
          074A    074E    0752
          0756    075A    075E
```

Nhận xét:

Địa chỉ của ma trận được cấp phát động thông qua con trỏ là:

(*A) = A[0] = *(A + 0) = & A[0][0] = 073E

&A[i][j] = (A[i] + j) = (*(A + i)) +j ;

****A[i][j] = *(A[i] + j) = *(*(A+i)) + j) ;***

Ví dụ 3.16: Viết chương trình tìm định thức của ma trận vuông bằng việc sử dụng con trỏ trỏ tới con trỏ.

```
#include      <stdio.h>
#include      <conio.h>
#include      <dos.h>
#include      <alloc.h>
#define             N       10
/* Khai báo nguyên mẫu cho hàm*/
void    Init_Matrix(float **A, int n);
void    In_Matrix(float **A, int n);
int             Chuyen(float **A, int n, int i);
```

```
float   Det_Matrix(float **A, int n);
/* Mô tả hàm */
void    Init_Matrix( float  **A, int n) {
        int i, j; float temp;
        for(i=0;i<n;i++){
                for(j=0;j<n; j++){
                        printf("\n Nhập A[%d][%d]=", i,j);
                        scanf("%f",&temp);
                        A[i][j]=temp;
                }
        }
        In_Matrix(A,n);
}

void    In_Matrix( float **A, int n){
        int i, j;
        for(i=0;i<n;i++){
                printf("\n%-3c", 179);
                for(j=0;j<n; j++){
                        printf("%8.2f", A[i][j]);
                }
                printf("%3c", 179);
        }
        delay(2000);
}
int chuyen(float **A, int n, int i){
        float j, k, temp;
        for(j=i+1; j<n;j++){
                if(A[j][i]){
                        for(k=0;k<n;k++){
                                temp=A[i][k];A[i][k]=A[j][k];
                                A[j][k]=temp;
                        }
```

PTIT

```
                return(1);
            }
        }
        return(0);
}
float Det_Matrix(float **A, int n){
        float D=1, p, x; int i, j, k;
        for(i=0; i<n;i++){
            if(A[i][i]==0){
                if(chuyen(A,n,i)==0){
                    printf("\n Định thức D= 0");
                    delay(2000);
                    return(0);
                }
                else
                    D =-D;
            }
            for(j=i+1;j<n;j++){
                x = A[j][i]/A[i][i];
                for(k=0; k<n;k++){
                    p = A[i][k] * x;
                    A[j][k]=A[j][k]-p;
                }
                In_Matrix(A,n); clrscr();
            }
        }
        for(i=0;i<n; i++)
            D *=A[i][i];
        printf("\n D=%6.2f", D);
        getch(); return(D);
}
void main(void){
        float *A; int n, i;clrscr();
```

PTIT

```
    printf("\n Nhập cấp n="); scanf("%d", &n);
    for ( i = 0; i <n ; i++)
        A[i] = malloc( (n-1) *sizeof(float));
    Init_Matrix(A,n);
    Det_Matrix(A,n);
    free(A);
}
```

5.6. Mảng các con trỏ

Mảng các con trỏ là một mảng mà mỗi phần tử của nó là một con trỏ. Vì con trỏ chính là biến nên phải có cách sử dụng mảng các con trỏ. Đây thực sự là một vấn đề cần xét và phân biệt giữa mảng các con trỏ và con trỏ trỏ tới con trỏ.

Khai báo cho mảng các con trỏ:

Kiểu_Dữ_Liệu * Tên_con_ trỏ[Số_Phần_Tử];

Ví dụ:

float *A[3]; khai báo mảng gồm 3 con trỏ float;

char *Word[20]; khai báo mảng gồm 20 con trỏ char;

Khởi đầu cho mảng con trỏ

Cũng giống như mảng nhiều chiều, mảng các con trỏ cũng có thể khởi đầu trực tiếp. Việc khởi đầu cho mảng con trỏ được minh hoạ thông qua hàm month_name(n) cho con trỏ tới xâu kí tự có chứa tên của tháng thứ n. Cú pháp của việc khởi đầu hoàn toàn tương tự với cách khởi đầu của mảng nhiều chiều:

```
char *month_name(n) /*cho tên của tháng thứ n*/
int n;
{
    char *name[ ] = {
        "tháng không hợp lệ"
        "Tháng giêng"
        "Tháng hai"
        "Tháng ba"
        "Tháng tư"
        "Tháng năm"
        "Tháng sáu"
        "Tháng bảy"
        "Tháng tám"
```

```
            "Tháng chín "
            "Tháng mười"
            "Tháng mười một"
            "Tháng mười hai"
        };
        return((n < 1 | | n > 12) ? name[0]: name[n]);
    }
```

Khai báo cho name, vốn là bảng các con trỏ kí tự, bộ khởi đầu chỉ là một danh sách các xâu kí tự; mỗi xâu được gán vào vị trí tương ứng trong bảng. Chính xác hơn, các kí tự của xâu thứ 1 được đặt ở đâu đó còn con trỏ tới chúng thì được lưu trữ trong name[i]. Vì kích thước của mảng name là không xác định nên bản thân trình biên dịch sẽ đếm các bộ khởi đầu và ghi vào số đúng.

Sự khác nhau giữa mảng hai chiều và mảng con trỏ, chẳng hạn như name trong ví dụ trên hoặc trong các khai báo

```
int A[10][10];
int *B[10];
```

việc sử dụng của A và B có thể là tương tự nhau, ở chỗ A[10][10] và *B[10] cả hai đều trỏ hợp lệ tới một bảng các phần tử int. Nhưng A thì là mảng cả 100 phần tử đều được cấp phát bộ nhớ một cách liên tục, việc tìm địa chỉ phần tử bất kỳ sẽ được thực hiện theo công thức. Tuy nhiên đối với B, khai báo chỉ cho phép cấp phát 10 con trỏ; mỗi con trỏ phải được đặt trỏ tới mảng số nguyên. Giả sử rằng mỗi con trỏ đều thực sự trỏ tới mảng 10 phần tử, vậy sẽ cần 100 vị trí lưu trữ phụ, cộng thêm 10 vị trí cho con trỏ. Do đó mảng con trỏ dùng tốn bộ nhớ hơn và còn có thể yêu cầu cả bước khởi đầu tường minh. Nhưng nó có hai ưu điểm: việc thâm nhập tới một phần tử được thực hiện gián tiếp thông qua con trỏ chứ không thực hiện các phép nhân và cộng, các hàng của mảng có thể có độ dài khác nhau. Tức là mỗi phần tử của b không nhất thiết trỏ tới vector 10 phần tử; đôi khi có thể trỏ tới hai phần tử, đôi khi 20 phần tử mà cũng có lúc chẳng trỏ tới phần tử nào.

5.7. Đối của hàm main()

Trong ngôn ngữ C, có một cách để truyền các đối hoặc tham biến trên dòng lệnh cho hàm main() ngay từ chương trình khi bắt đầu thực hiện. Khi hàm main() được kích hoạt, nó sẽ gọi tới hai đối. Đối thứ nhất là một biến nguyên được qui định là argc, argc là số các đối trên dòng lệnh mà chương trình sẽ tạo ra; đối thứ hai được qui định là argv, argv là con trỏ tới mảng các xâu kí tự, mỗi xâu kí tự trong mảng các xâu kí tự là một đối. Việc thao tác các xâu kí tự này phụ thuộc vào mục tiêu của người sử dụng. Đối thứ nhất của argv (argv[0]) được qui định là tên của chương trình. Ví dụ trong câu lệnh copy của DOS:

copy vanban1.txt vanban2.txt

Thì copy là tên của chương trình cần thực hiện, đối thứ nhất của hàm main là argc có giá trị là 3, đối thứ nhất là argv[0] = "copy", đối thứ hai là

argv[1]=”vanban1.txt”, đối thứ hai là argv[2]=”vanban2.txt”. Các đối tuỳ chọn được phép viết theo bất kỳ trật tự nào, phần còn lại của chương trình sẽ tự động cảm nhận giá trị của argc với số các đối thực tế có mặt.

Chương trình sau là một minh hoạ đơn giản nhất về các khai báo cần thiết và cách sử dụng đối của hàm main(). Chương trình này chỉ cho hiện lại các đối dòng lệnh của nó trên một dòng, mỗi đối cách nhau bởi các dấu trống. Tức là nếu ta có dòng lệnh

echo hello, world

thì chương trình sẽ cho ra

hello, world

Theo qui ước, argv[0] là tên chương trình, vậy argc ít nhất là 1. Trong ví dụ trên, argc là 3 và argv[0], argv[1] và argv[2] tương ứng là “echo”, “hello” và “world”. Đối thực sự đầu tiên là argv[1] và cuối cùng là argv[argc- 1]. Nếu argc là 1 thì sẽ không có đối dòng lệnh nào theo sau tên chương trình. Điều này được trình bày trong chương trình echo.c:

Ví dụ 3.17: In ra các đối của hàm main()

```
#include    <stdio.h>
void main(int argc, char *argv[] ) /*hiện các đối của main*/
{
        int i;
        for (i = 1; i < argc; i + +)
            printf(“%10s ,argv[i]\ n”);
}
```

Ví dụ 3.18: Viết chương trình tính tổng của n số thực với đối số được truyền trực tiếp cho hàm main().

```
#include      <stdio.h>
float main( int argc, char *argv[] ) {
      float   s =0, k; int i;
      for ( i =1; i< argc; i++){
            k = atof(argv[i]);
            s += k;
      }
      printf(“Tổng s = %6.2f”, s);
      return(s);
}
```

Kết quả thực hiện chương trình:

```
TONG        1 2 3 4 5 6 7 8 9
Tổng s = 45.00
```

Thí dụ 3.19: Viết chương trình sắp xếp tất cả các đối của hàm main theo thứ tự tăng dần.

```
#include    <stdio.h>
#include    <string.h>
void main( int argc, char *argv[]){
        int i, j; char  *temp;
        for(i=1;i<argc-1;i++){
                for(j = i+1; j< argc; j++){
                        if (strcmp(argv[i], argv[j])>0){
                                temp=argv[i]; argv[i]= argv[j];
                                argv[j] = temp;
                        }
                }
        }
        for(i=1; i< argc; i++)
                printf("%-10s", argv[i]);
}
```

Kết quả thực hiện chương trình:

```
SAPXEP 9  8 7  6  5  4  3  2 1 0
```

5.8. Con trỏ hàm

Trong C, bản thân hàm không phải là một biến, nhưng có thể định nghĩa một con trỏ tới hàm, thông qua con trỏ hàm chúng ta có thể coi địa chỉ hàm như một biến, có thể được truyền cho hàm, được đặt vào bảng. Điều quan trọng là chúng ta có thể sử dụng con trỏ hàm để quản lý và thực hiện mọi hàm có cùng kiểu với nó và làm tăng đáng kể tính mềm dẻo của hàm. Để hiểu rõ bản chất của con trỏ hàm chúng ta cần phải biết rõ về cơ chế dịch chương trình và lời gọi hàm trong C. Khi dịch chương trình, chương trình dịch trong C chuyển các hàm từ chương trình nguồn thành dạng mã OBJ, mỗi hàm được coi như một modul được cấp phát bộ nhớ tại một vị trí nào đó gọi là địa chỉ của hàm, địa chỉ của hàm là điểm đầu thực hiện của hàm mà tại đó hàm nhận được đầy đủ những tham biến cần thiết và thực hiện hàm.

Khi có một lời gọi hàm trong khi chương trình thực hiện, ngôn ngữ lập trình chuyển điều khiển chương trình tới địa chỉ của hàm tương ứng và thực hiện lời gọi hàm từ chính địa chỉ này. Chương trình dịch của C qui định, tên của hàm đồng thời

là địa chỉ của hàm trong bộ nhớ, do vậy ta hoàn toàn có thể thực hiện được phép gán con trỏ hàm bằng giá trị là tên của một hàm. Việc khai báo một con trỏ hàm được thực hiện thông qua dòng khai báo sau:

Kiểu_dữ_liệu (*Tên_Hàm)();

Ví dụ:

int (*cmp)(); khai báo con trỏ hàm kiểu int;

float (*fcmp)();khai báo con trỏ hàm kiểu float;

Ví dụ 3.20: Xây dựng con trỏ hàm đơn giản tính tổng, hiệu, tích, modul của hai số nguyên dương a, b;

```
#include <stdio.h>
int tong( int a, int b) { return(a+b);}
int hieu( int a, int b) { return(a-b);}
int tich( int a, int b) { return(a*b);}
int modul( int a, int b) { return(a%b);}
void main(void) {
    int (*cmp)(); /* khai báo con trỏ hàm kiểu int */
    int a, b;
    printf("\n Nhập a="); scanf("%d", &a);
    printf("\n Nhập b="); scanf("%d", &b);
    cmp=tong;
    printf("\n Tổng a + b =%d", cmp(a,b));
    cmp = hieu;
    printf("\n Hiệu a - b =%d", cmp(a,b));
    cmp = tich;
    printf("\n Tích a * b =%d", cmp(a,b));
   cmp = modul;
    printf("\n modul(a, b) =%d", cmp(a,b));
    getch();
}
```

Nhận xét: Trong ví dụ trên, con trỏ hàm cmp lần lượt nhận địa chỉ của các hàm tong, hieu, tich, modul và thực hiện các hàm này giống như các nguyên mẫu của nó. Tổng quát, tất cả các hàm có cùng kiểu với cmp đều có thể được quản lý bằng một con trỏ hàm.

PTIT

Trong ví dụ sau, để tìm bội số chung nhỏ nhất của hai số nguyên dương a và b chúng ta sẽ sử dụng con trỏ hàm như một biến để truyền cho hàm. Vì bội số chung nhỏ nhất của a và b là thương số của tích a và b cho ước số chung nhỏ nhất của a và b. Do đó, ta hoàn toàn có thể thiết kế được một hàm bao gồm các đối số là a, b và một con trỏ hàm là ước số chung nhỏ nhất của a, b. Chương trình được thể hiện như sau:

Ví dụ 3.21: Tìm bội số chung nhỏ nhất của hai số nguyên dương a, b;

```
#include      <stdio.h>
int     USCLN( int  a, int b) {
        while ( a!=b){
                if(a > b)  a  = a-b;
                else     b = b-a;
        }
        return(a);
}
int     BSCNN( int a, int b, int  (*cmp)() ) {
        int k = cmp(a,b);
        return( (a * b)/k);
}
void main(void) {
        int  a, b, (*p)();
        printf("\n Nhập a ="); scanf("%d", &a);
        printf("\n Nhập b ="); scanf("%d", &b);
        p = USCLN;
        printf("\n BSCNN = %d", BSCNN(a, b, p));
        getch();
}
```

PTIT

6. Dữ liệu kiểu tệp (FILE)

6.1. Thâm nhập vào thư viện chuẩn

Mỗi tệp gốc có tham trỏ tới hàm thư viện chuẩn đều phải chứa dòng khai báo

#include < tên_tệp_thư_viện >